AQI Predictor Project

Milestone 2 Report: Feature Pipeline Script

Ayesha Salahuddin
GitHub: github.com/ayeshasalahuddin/AQI-predictor

***Abstract*—This report documents the design, implementation, and verification of the Feature Pipeline for the Pearls AQI Predictor system. The feature pipeline is the foundational data layer of the end-to-end machine learning system. It fetches real-time Air Quality Index (AQI) and meteorological data from the AQICN API for Karachi, engineers time-based and derived features, and stores them as versioned rows in the Hopsworks Feature Store. Successful execution of this pipeline was verified by inserting live data with AQI value 161 into the cloud Feature Store, with a materialization job confirmed as launched on the Hopsworks platform.**

***Index Terms*—AQI, feature pipeline, feature store, Hopsworks, AQICN, feature engineering, machine learning, air quality**

I. Introduction

Milestone 2 builds the core data layer of the AQI Predictor system: the Feature Pipeline. This pipeline is responsible for collecting raw environmental data, transforming it into ML-ready features, and persisting it in a centralized Feature Store for downstream model training and inference.

The feature pipeline is the most critical component of the system because every subsequent milestone — model training, automated scheduling, and the web dashboard — depends entirely on the quality and availability of data produced here. Without a reliable, well-engineered feature pipeline, no machine learning can take place.

This milestone achieves the following:

- Live data ingestion from the AQICN pollution API
- Feature engineering including time-based and categorical features
- Secure credential management via environment variables
- Persistent cloud storage via the Hopsworks Feature Store
- End-to-end verification with real Karachi AQI data

II. System Architecture

The feature pipeline sits at the entry point of the full ML system architecture. Fig. 1 illustrates the data flow from the external API through the pipeline to the Feature Store.

AQICN API → **feature_pipeline.py** → **Hopswork**
(Raw data) (Fetch + Engineer) (Feature Sto

The pipeline is designed to be stateless and idempotent — it can be run at any time, any number of times, and will always produce a correctly formatted row for the Feature Store. This property is essential for the automated hourly scheduling that will be implemented in Milestone 6.

III. Implementation

A. Script Structure

The feature pipeline is implemented as a single Python script `feature_pipeline.py` organized into four functions:

- `fetch_raw_data()` — calls the AQICN API and extracts pollutant and weather values
- `engineer_features(raw)` — computes all engineered features from raw data
- `get_aqi_category(aqi)` — maps numeric AQI to a health category string
- `store_to_feature_store(features)` — connects to Hopsworks and inserts the feature row

B. Data Ingestion

Raw data is fetched from the AQICN API endpoint:

```
url = f"https://api.waqi.info/feed/{CITY}/?
    token={TOKEN}"
response = requests.get(url)
data = response.json()
```

The API returns a nested JSON object. The `iaqi` field contains individual pollutant readings. Each pollutant value is extracted with a safe fallback to `None` if not available:

```
iaqi = d.get("iaqi", {})
raw = {
"aqi":          d.get("aqi", None),
"pm25":         iaqi.get("pm25",{}).get("v",
    None),
"pm10":         iaqi.get("pm10",{}).get("v",
    None),
"no2":          iaqi.get("no2", {}).get("v",
    None),
"o3":           iaqi.get("o3",  {}).get("v",
    None),
"temperature": iaqi.get("t",   {}).get("v",
    None),
"humidity":     iaqi.get("h",   {}).get("v",
    None),
"wind":         iaqi.get("w",   {}).get("v",
    None),
"pressure":     iaqi.get("p",   {}).get("v",
    None),
}
```

C. Feature Engineering

Feature engineering transforms raw API values into structured ML inputs. Two categories of features are computed:

1) Time-Based Features: Time-based features capture cyclical patterns in air quality. Research shows AQI varies significantly by hour of day (rush hour peaks), day of week (weekday vs weekend traffic), and month (seasonal weather patterns).

- `hour` — hour of day (0–23)
- `day_of_week` — day of week (0=Monday, 6=Sunday)
- `month` — month of year (1–12)
- `is_weekend` — binary flag (1 if Saturday or Sunday)

2) Pollutant Features: All raw pollutant readings are cast to float with a zero fallback for missing values, ensuring the Feature Store always receives a consistent schema:

- `aqi` — overall Air Quality Index
- `pm25` — fine particulate matter ($\leq$2.5μm)
- `pm10` — coarse particulate matter ($\leq$10μm)
- `no2` — nitrogen dioxide
- `o3` — ground-level ozone
- `so2` — sulphur dioxide
- `co` — carbon monoxide
- `temperature` — ambient temperature (°C)
- `humidity` — relative humidity (%)
- `wind` — wind speed (m/s)
- `pressure` — atmospheric pressure (hPa)

3) AQI Category Feature: A categorical feature `aqi_category` is derived from the numeric AQI using the standard US EPA scale:

AQI Range	Category
0 – 50	good
51 – 100	moderate
101 – 150	unhealthy_sensitive
151 – 200	unhealthy
201 – 300	very_unhealthy
301+	hazardous

D. Hopsworks Feature Store Integration

The pipeline connects to Hopsworks using the Python SDK and inserts features into a versioned Feature Group:

```
project = hopsworks.login(
    api_key_value=HOPSWORKS_API_KEY,
    project="PredictorAQI"
)
fs = project.get_feature_store()

fg = fs.get_or_create_feature_group(
    name="aqi_features",
    version=1,
    primary_key=["city", "timestamp"],
    description="Hourly AQI features for
        Karachi",
    event_time="timestamp",
)
fg.insert(df)
```

The `get_or_create_feature_group()` call is idempotent — on first run it creates the Feature Group with the correct schema; on subsequent runs it retrieves the existing group and appends the new row. The primary key is a composite of `city` and `timestamp`, ensuring no duplicate entries.

E. Credential Management

All API keys are stored in a `.env` file excluded from version control via `.gitignore`. The `python-dotenv` library loads them at runtime:

```
load_dotenv()
AQICN_TOKEN = os.getenv("AQICN_TOKEN")
HOPSWORKS_API_KEY = os.getenv("
    HOPSWORKS_API_KEY")
```

This ensures credentials are never exposed in the codebase or on GitHub.

IV. Technical Challenges and Resolutions

Several Windows-specific issues were encountered and resolved during implementation:

Issue	Cause	Resolution
`/tmp` path not found	Hopsworks uses Linux path hardcoded	Created `C:\tmp` and set `TEMP` env variable
`confluent-kafka` missing	Not included in default install	Installed via pip separately
Version mismatch 4.8.1 vs 4.7.2	Latest hopsworks client ahead of backend	Downgraded to `hopsworks==4.7.*`
pandas build failure	Missing Ninja/CMake build tools on Windows	Used Anaconda pre-compiled binaries

V. Results and Verification

The pipeline was executed successfully and produced the following terminal output:

```
AQI Feature Pipeline Starting...
Fetched AQI: 161 at 2026-04-28 21:07:39
aqi: 161.0 | pm25: 161.0 | temperature: 31.0
humidity: 3.0 | wind: 5.1 | pressure: 1016.0
hour: 21 | day_of_week: 1 | month: 4
is_weekend: 0 | aqi_category: unhealthy
Connecting to Hopsworks...
Python Engine initialized.
Logged in to project PredictorAQI
Uploading Dataframe: 100% -- Rows 1/1
Job started: aqi_features_1_offline_fg_materialization
Successfully inserted row into Feature Store.
Pipeline complete.
```

The Feature Group `aqi_features` was created in the Hopsworks Feature Store under project `PredictorAQI`. The materialization job was confirmed as launched, meaning the data is being persisted to offline storage for use in model training.

Key observations from the output:

- AQI of 161 places Karachi in the **Unhealthy** category

- Humidity reading of 3% is unusually low, suggesting a dry desert wind event
- PM2.5 equals AQI (161), indicating fine particulate matter is the dominant pollutant
- All time-based features were computed correctly from UTC timestamp

VI. Feature Summary

The table below lists all features stored in the Hopsworks Feature Store after Milestone 2:

Feature	Type	Source
aqi	float	AQICN API
pm25	float	AQICN API
pm10	float	AQICN API
no2	float	AQICN API
o3	float	AQICN API
so2	float	AQICN API
co	float	AQICN API
temperature	float	AQICN API
humidity	float	AQICN API
wind	float	AQICN API
pressure	float	AQICN API
hour	int	Engineered
day_of_week	int	Engineered
month	int	Engineered
is_weekend	int	Engineered
aqi_category	string	Engineered
timestamp	datetime	System
city	string	Config

VII. Next Steps — Milestone 3

Milestone 3 will implement the Historical Data Backfill. The feature pipeline will be run across a range of past dates (6–12 months) to populate the Feature Store with thousands of historical rows. This historical dataset forms the training data for all ML models in Milestone 5.

Key tasks in Milestone 3:

- Write `backfill.py` script that loops over a date range
- Use the AQICN historical feed endpoint for past data
- Insert all historical rows into the existing `aqi_features` Feature Group
- Verify thousands of rows appear in the Hopsworks Data Preview

VIII. Conclusion

Milestone 2 successfully delivered a fully functional, cloud-connected feature pipeline for the AQI Predictor system. The pipeline fetches live pollution and weather data, engineers 18 structured features, and persists them to the Hopsworks Feature Store. All Windows-specific compatibility issues were resolved. The system is now ready for historical backfilling in Milestone 3 and model training in Milestone 5.